



มหาวิทยาลัยเอเชียอาคเนย์

ระดับ ☒ ปริญญาตรี ☐ ปริญญาโท ☐ ปริญญาเอก คณะบริหารธุรกิจ

หลักสูตร/สาขาวิชา สาขาวิชาสารสนเทศเพื่อธุรกิจดิจิทัล ชั้นปีที่ 1

ข้อสอบ ☒ กลางภาค ☐ ปลายภาค ประจำปีการศึกษาที่ 2 ปีการศึกษา 2568 ภาคปกติ

รหัสวิชา 725103 ชื่อวิชา โครงสร้างข้อมูลและอัลกอริทึม จำนวน 3 หน่วยกิต คะแนนเต็ม 105 คะแนน

สอบวันที่ เวลา รวม 2 ชั่วโมง (ข้อสอบมีทั้งหมด 8 หน้า)

คำสั่ง

- ให้นักศึกษาทำข้อสอบลงใน ☐ สมุดคำตอบ ☒ ข้อสอบ ☐ กระดาษคำตอบ
- ☒ อนุญาตเปิดเอกสารระหว่างการสอบ ☐ ไม่อนุญาตให้นำเอกสารทุกชนิดเข้าห้องสอบ
☒ อื่นๆ (ระบุ) จดเข้าห้องสอบ 1 แผ่น A4
- ข้อสอบทั้งหมดมีทั้งหมด 1 ตอน 105 คะแนน (เต็ม 100 คะแนน)
- ข้อสอบ ☐ มี ☒ ไม่มี ตาราง/สูตรคำนวณ/เอกสารประกอบ จำนวน แผ่น/เล่ม
- ☐ อนุญาต ☒ ไม่อนุญาต ให้ใช้เครื่องคำนวณทุกชนิด
- ☐ อนุญาต ☒ ไม่อนุญาต ใช้ไม้บรรทัดที่มีสูตรเข้าห้องสอบ
- ข้อสอบถือว่าสมบูรณ์ หากมีข้อผิดพลาดให้ใช้ดุลพินิจเองและให้แจ้งใน Microsoft Teams

.....

ผู้ออกข้อสอบ
อาจารย์ปพนธิ์ แยมโชติ
เบอร์โทร 087-049-2475

1. Data in Memory & Arrays (15 คะแนน)

- (a) (6 คะแนน) อธิบายว่า **Array เก็บข้อมูลใน memory** มีลักษณะอย่างไร (เช่น contiguous memory, index $\rightarrow$ address) และเหตุผลที่ทำให้การเข้าถึง $A[i]$ เป็นเวลา $O(1)$

แนวคำตอบ (ตัวอย่าง):

- **Array** เก็บข้อมูลแบบ **contiguous memory** คือสมาชิกอยู่ ติดกันในหน่วยความจำ ทีละช่อง (slot) ขนาดเท่ากัน
- การอ้างอิงสมาชิกด้วย index ทำได้โดยคำนวณ address จาก **base address**:

$$\text{address}(A[i]) = \text{base} + i \times \text{size_of_element}$$

- ดังนั้นการเข้าถึง $A[i]$ ไม่ต้องไล่ทีละตัว แต่คำนวณตำแหน่งได้ทันที จึงเป็นเวลา $O(1)$

- (b) (6 คะแนน) กำหนดให้ int ใช้ 4 bytes และ $A[0]$ มี address = 1200 จงหา address ของ $A[13]$ พร้อมแสดงสูตร

$$\text{address}(A[i]) = \text{address}(A[0]) + i \times 4$$

$$\therefore \text{address}(A[13]) = 1200 + 13 \times 4 = 1200 + 52 = \boxed{1252}$$

- (c) (3 คะแนน) ยกตัวอย่าง ข้อดี 1 อย่าง และ ข้อเสีย 1 อย่าง ของ Array ในมุมมองการจัดการ memory/การแทรก-ลบ

ข้อดี: เข้าถึงด้วย index ได้ทันที $O(1)$ (random access) และมี cache locality ดี

ข้อเสีย: การ insert/delete ตรงกลางต้อง shift สมาชิกจำนวนมาก ทำให้เป็น $O(n)$ และการขยาย (resize) อาจต้องสร้างใหม่หรือย้ายไป block ใหม่ที่ติดกัน

2. Abstract Data Types (ADT) (12 คะแนน)

- (a) (6 คะแนน) ADT คืออะไร? อธิบายให้ชัดเจนว่า “บอกอะไร” และ “ไม่บอกอะไร”

ADT (Abstract Data Type) คือการกำหนด “สเปก/พฤติกรรม” ของชนิดข้อมูลผ่าน ชุดของ **operations** และความหมายของผลลัพธ์ (semantics)

ADT บอก:

- มี operation อะไรบ้าง (เช่น push, pop, peek)
- แต่ละ operation ทำงานอย่างไรในเชิงพฤติกรรม/กฎ (เช่น LIFO, คำนวณอะไรเมื่อว่าง)

ADT ไม่บอก:

- ไม่บอกว่าจะเก็บใน memory แบบไหน (array/linked list)
- ไม่บอกโค้ด/โครงสร้างหรือ logic ภายในที่ใช้ implement จริง

- (b) (3 คะแนน) อธิบายความต่างระหว่าง ADT กับ Data Structure

ADT คือ WHAT (นิยามว่าทำอะไร/พฤติกรรมเป็นอย่างไร)

Data Structure คือ HOW (วิธี implement จริง เช่น ใช้ array หรือ linked list)

ตัวอย่าง: **Stack (ADT)** สามารถ implement ด้วย **Dynamic Array** หรือ **Linked List** ก็ได้

- (c) (3 คะแนน) ยกตัวอย่าง **Stack** เป็น ADT โดยระบุ operation หลักอย่างน้อย 3 รายการ และบอกพฤติกรรมสำคัญ

ตัวอย่าง operations ของ **Stack ADT**:

- **push(x)**: ใส่ข้อมูล x ลงบนสุด
- **pop()**: นำข้อมูลบนสุดออกและคืนค่านั้น
- **peek()**: ดูข้อมูลบนสุดแต่ไม่เอาออก
- (เสริม) **isEmpty()**, **size()**

พฤติกรรมสำคัญ: **LIFO (Last-In First-Out)** ใส่ล่าสุดจะถูกเอาออกก่อน

3. (6 คะแนน) จงเติมตารางฟังก์ชันสำหรับวัดผล Big-O ต่อไปนี้เรียงจากซับซ้อนน้อยสุด ไปซับซ้อนมากที่สุด

1						$n!$
---	--	--	--	--	--	------

$$1 < \log n < n < n \log n < n^k < c^n < n!$$

4. Pseudocode Tracing (18 คะแนน)

3.1 Trace ค่าในรูป (9 คะแนน)

ให้ $A = [3, 1, 4, 1, 5]$ และ $n = 5$ พิจารณา pseudocode:

```

1:  $sum \leftarrow 0$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:   if  $A[i]$  is odd then
4:      $sum \leftarrow sum + A[i]$ 
5:   end if
6: end for
7: return  $sum$ 

```

จงเขียนค่าของ i และ sum หลังจบแต่ละรอบ และตอบค่า **return** สุดท้าย

รอบที่	i	$A[i]$ (odd?)	sum หลังจบรอบ
1	0		
2	1		
3	2		
4	3		
5	4		

ค่า **return** สุดท้ายคือ: _____

รอบที่	i	$A[i]$ (odd?)	sum หลังจบรอบ
1	0	3 (yes)	3
2	1	1 (yes)	4
3	2	4 (no)	4
4	3	1 (yes)	5
5	4	5 (yes)	10

ค่า return สุดท้ายคือ: 10

3.2 Trace Binary Search (9 คะแนน)

ให้ array ที่เรียงแล้ว $B = [2, 4, 7, 10, 12, 15, 20, 27]$ (index 0..7) ต้องการหา $key = 15$

จงแสดงตารางการทำงานของ low, high, mid ที่ละรอบจนกว่าจะเจอ (หรือจบการค้นหา) โดยใช้ $mid = (n+1)/2$ แบบปัดเศษลงเสมอ

รอบที่	low index	high index	mid index	$B[mid]$	ผลการเปรียบเทียบ/อัปเดต
1					
2					
3					
4					
5					
6					
7					
8					

รอบที่	low index	high index	mid index	$B[mid]$	ผลการเปรียบเทียบ/อัปเดต
1	0	7	3	10	$10 < 15 \Rightarrow low \leftarrow 4$
2	4	7	5	15	พบ key ที่ index 5 (จบ)

สรุป: พบค่า 15 ที่ index = 5

5. Big-O จากการนับจำนวนรอบ/จำนวน operation (15 คะแนน)

พิจารณา pseudocode:

```

1:  $count \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $j \leftarrow 1$ 
4:   while  $j \leq n$  do
5:      $count \leftarrow count + 1$ 
6:      $j \leftarrow j * 2$ 
7:   end while
8: end for
9: return  $count$ 

```

(a) (6 คะแนน) ในหนึ่งรอบของ for ลูป while ทำงานกี่รอบ (เขียนเป็นฟังก์ชันของ n)

ค่า j เริ่มที่ 1 แล้วคูณ 2 ไปเรื่อย ๆ: 1, 2, 4, 8, ...
 ให้จำนวนรอบของ while เท่ากับ t จะมี $j = 2^{t-1}$ ในรอบสุดท้ายที่ยังทำงานอยู่ และต้องมี

$$2^{t-1} \leq n < 2^t$$

ดังนั้น $t = \lfloor \log_2 n \rfloor + 1$

สรุป: while ทำงาน $\lfloor \log_2 n \rfloor + 1$ รอบ ต่อหนึ่งรอบของ for

(b) (6 คะแนน) จงหาค่า $count$ ตอนจบ (แสดงเป็นฟังก์ชันของ n)

ในแต่ละรอบของ for ค่า $count$ จะเพิ่มขึ้นเท่ากับจำนวนรอบของ while
 ดังนั้นเมื่อ for ทำงาน n รอบ:

$$count = n (\lfloor \log_2 n \rfloor + 1)$$

(ถ้าละ floor เพื่อสื่อภาพรวม จะได้ประมาณ $n \log_2 n + n$)

(c) (3 คะแนน) จงให้ค่าเวลาเชิงซ้อนเป็น Big-O

$$count = n(\lfloor \log_2 n \rfloor + 1) \Rightarrow \text{เวลา} = O(n \log n)$$

คำตอบ: $O(n \log n)$

6. Ternary Search (12 คะแนน)

นักศึกษาทุกคนควรจะเข้าใจ Binary Search กันอยู่แล้วว่าเป็น algorithm ที่ค้นหาสมาชิกที่ให้มาใน array ที่มีการเรียงค่าจากน้อยไปมาก โดยแนวคิดหลักคือดูตัวตรงกลางก่อนว่าใช้ตัวที่กำลังหาหรือไม่ ถ้าไม่ใช่ค่อยแบ่งครึ่งออกด้วยสมาชิกตัวตรงกลางนั้นออกเป็นฝั่งน้อยกว่าและฝั่งมากกว่า และทำแบบนี้ไปเรื่อย ๆ จนเหลือสมาชิกตัวเดียวที่จะแบ่งออกต่อไปไม่ได้แล้ว คราวนี้จึงเกิดคำถามว่าถ้าเราแบ่งเป็น 3 ส่วน algorithm จะซับซ้อนน้อยลงหรือไม่ (เตือนอีกครั้งว่าไม่ใช่การถามว่าเร็วขึ้นหรือไม่) โดยเสนอแนวคิดที่ใช้สมาชิก 2 ตัวแบ่ง array ออกเป็น 3 ส่วน ตรวจสอบก่อนว่าสมาชิกที่กำลังหาอยู่ใน 2 ตัวที่เป็นตัวแบ่ง array หรือไม่ ถ้าไม่ใช่ค่อยลงไปหาใน array ที่ขึ้นส่วนใดขึ้นส่วนหนึ่งจาก 3 ขึ้นส่วนที่แบ่งออกมา

- (a) (6 คะแนน) จงเขียน pseudocode ของ Ternary Search ตามแนวคิดที่กล่าวไป ที่รับ input เป็น array ที่เรียง-
ค่าจากน้อยไปมากแล้ว และ key ซึ่งคือค่าที่กำลังหา แล้วคืนค่า “index ที่พบ” หรือ -1 ถ้าไม่พบ

```

1: function TernarySearch(A, key)                                ▷ A sorted ascending
2:   low ← 0
3:   high ← length(A) - 1
4:   while low ≤ high do
5:     third ← ⌊(high - low)/3⌋
6:     mid1 ← low + third
7:     mid2 ← high - third
8:     if A[mid1] = key then
9:       return mid1
10:    else if A[mid2] = key then
11:      return mid2
12:    else if key < A[mid1] then
13:      high ← mid1 - 1
14:    else if key > A[mid2] then
15:      low ← mid2 + 1
16:    else
17:      low ← mid1 + 1
18:      high ← mid2 - 1
19:    end if
20:  end while
21:  return -1
22: end function

```

- (b) (6 คะแนน) จงวิเคราะห์ Big-O time complexity ของ Ternary Search

วิเคราะห์แบบ iteration (นับจำนวนรอบ):

ในแต่ละรอบของ Ternary Search เราเปรียบเทียบค่าคงที่ (เช่น ตรวจ mid1, mid2 และเลือกช่วง) ดังนั้น
ค่าใช้จ่ายต่อรอบเป็น $O(1)$

และช่วงค้นหาจะถูกลดจาก n เหลือประมาณ $n/3$ ในรอบถัดไป

ให้ k = จำนวนรอบที่ทำงานจนช่วงเหลือขนาด 1

หลังทำไป k รอบ ขนาดช่วงจะเหลือประมาณ

$$\frac{n}{3^k}$$

หยุดเมื่อ $\frac{n}{3^k} \leq 1$

$$\frac{n}{3^k} \leq 1 \Rightarrow n \leq 3^k \Rightarrow k \geq \log_3 n$$

ดังนั้นจำนวนรอบเป็น $k = \lceil \log_3 n \rceil$ และแต่ละรอบใช้เวลา $O(1)$

สรุปเวลาเชิงซ้อนคือ

$$T(n) = O(\log_3 n) = \boxed{O(\log n)}$$

7. Dynamic Array vs Linked List (12 คะแนน)

อธิบายเปรียบเทียบ Dynamic Array กับ Linked List ในหัวข้อต่อไปนี้

หัวข้อ	Dynamic Array	Linked List
(ก) get(i) ใช้เวลาเท่าไร และทำไม (1 คะแนน)		
(ข) append ใช้เวลาเท่าไร (พูดถึง <i>amortized</i>) (1 คะแนน)		
(ค) insert/delete ที่ “หน้าสุด” ใช้เวลาเท่าไร (1 คะแนน)		
(ง) memory overhead เปรียบเทียบ (1 คะแนน)		

หัวข้อ	Dynamic Array	Linked List
(ก) get(i) ใช้เวลาเท่าไร และทำไม (1 คะแนน)	$O(1)$ เพราะคำนวณ address ได้ทันที	$O(n)$ เพราะต้องไล่ node ทีละตัวจนถึง index i
(ข) append ใช้เวลาเท่าไร (พูดถึง <i>amortized</i>) (1 คะแนน)	<i>amortized</i> $O(1)$ (บางครั้ง resize เป็น $O(n)$)	$O(1)$ ถ้ามี tail pointer (ถ้าไม่มี tail อาจเป็น $O(n)$)
(ค) insert/delete ที่ “หน้าสุด” ใช้เวลาเท่าไร (1 คะแนน)	$O(n)$ เพราะต้อง shift สมาชิก	$O(1)$ (ปรับ pointer ที่หัว list)
(ง) memory overhead เปรียบเทียบ (1 คะแนน)	overhead ต่อ element น้อย แต่มี capacity เผื่อ	overhead สูงกว่าเพราะมี pointer/metadata ต่อ node และกระจาย memory

8. Stack & Queue (6 คะแนน)

เลือกโครงสร้างข้อมูลที่เหมาะสม (Stack หรือ Queue) พร้อมเหตุผลสั้นๆ

- (a) (3 คะแนน) ระบบปุ่ม “ย้อนกลับ (Back)” ของเว็บเบราว์เซอร์

คำตอบ: Stack

เหตุผล: เป็นพฤติกรรมแบบ LIFO หน้าทีเข้าล่าสุดควรถูกย้อนกลับก่อน

- (b) (3 คะแนน) ระบบจัดคิวงานพิมพ์ (Printer jobs) มาก่อนทำก่อน

คำตอบ: Queue

เหตุผล: เป็นพฤติกรรมแบบ FIFO งานที่มาก่อนควรถูกประมวลผลก่อน

9. (9 คะแนน) สมมติว่าในภาษาโปรแกรมที่นักศึกษา กำลังจะใช้ไม่มี Stack ให้ใช้ นักศึกษาจึงต้องทำการสร้าง Stack ขึ้นมาใช้งานเอง แต่นักศึกษานึกขึ้นมาได้ว่ามีคนเก่าในทีมเคยทำ Linked List ขึ้นมาแล้ว และเพราะว่าไม่อยากจะยุ่งวุ่นวายกับการจัดการหน่วยความจำอีกครั้ง จึงตกลงกับทีมว่าจะนำ Linked List นั้นมาใช้เป็นฐานเพื่อสร้าง Stack ขึ้นมา โดย ADT ของ Linked List ที่นักศึกษาเคยสร้างไว้คือ

LinkedList:

- Insertion(x, idx): แทรกสมาชิก x ไปที่ index ที่ idx และไม่คืนค่าอะไรออกมา
- Deletion(idx): ลบสมาชิกที่ index ที่ idx และไม่คืนค่าอะไรออกมา
- Get(idx): ดูว่าสมาชิกที่เก็บอยู่ที่ index ที่ idx คือเลขอะไร และคืนค่าสมาชิกที่อยู่ index นั้นออกมา
- Searching(x): หาว่ามีสมาชิก x อยู่ใน Linked List หรือไม่ และคืนค่า index ที่เจอสมาชิกตัวนั้น หรือ -1 ในกรณีไม่เจอ
- Updating(x, idx): เปลี่ยนสมาชิกที่ index ที่ idx ให้เป็น x
- Length(): ดูว่า Linked List ที่ใช้อยู่มีสมาชิกอยู่กี่ตัว

ให้นักศึกษาใช้ methods ที่มีใน LinkedList ตามที่นำมาเพื่อมาสร้าง methods 'push(x)' เพื่อใส่สมาชิก x ลงไปใน stack และ 'pop()' เพื่อดึงสมาชิกบนสุดออกมา โดยที่ถ้าไม่มีสมาชิกให้คืนค่า None

(a) Pseudocode ของ Push(x)

แนวคิด: ให้ **top** ของ stack อยู่ที่ index 0 (หัว list) จะทำ push/pop ได้ง่าย

```
1: function push(x)
2:   Insertion(x, 0)
3: end function
```

(b) Pseudocode ของ Pop()

```
1: function pop
2:   if Length = 0 then
3:     return None
4:   end if
5:   val ← Get(0)
6:   Deletion(0)
7:   return val
8: end function
```